

Summary and Analysis: SwAV

Moataz Hamza

Academic College of Tel Aviv–Yaffo

Abstract

SwAV is my introduction to a new paradigm of self-supervised learning: clustering-based methods! Unlike prior clustering approaches, SwAV operates as an online method while borrowing key ideas from contrastive learning frameworks like SimCLR (which I analyzed in my previous post). In this post, I break down SwAV's core architecture, mathematical formulation, and empirical results. I also revisit the false-negative problem discussed previously to examine how SwAV bypasses it.

1 Same Problem, Different Approach

This paper addresses the same problem as the previous two papers I analyzed (SimCLR, MoCo): self-supervised visual representation learning. The difference is in how they try to solve it.

Clustering-based Approach. Clustering-based methods, just like other self-supervised methods, also create pseudo-labels that we can use for pretext tasks to learn visual representations, and as you can guess from the name those pseudo-labels are cluster assignments. This is how it works:

- **Pseudo-labels creation:** first, we assign each image to a cluster, using some clustering algorithm.
- **Pretext task:** then we train a model to predict the cluster assignment of a given image.

And by solving this task, we hope that our model learns useful visual representations.

Contrastive Learning. In SwAV they criticize the fact that these clustering approaches are not *online*: You have to go over the whole dataset twice, once to create the pseudo-labels and the second time to solve the pretext task. In contrast, contrastive learning approaches create their labels on the go, e.g., SimCLR and MoCo consider each image and its different views one class. But these methods introduce a computational challenge: they rely on a large number of explicit pairwise feature comparisons.

SwAV introduces an online clustering-based self-supervised method that combines elements and ideas from the previous two methods. They describe their method as "*contrasting between multiple image views by comparing their cluster assignments*". So unlike clustering-based methods, they are not treating the cluster assignments as labels, instead they use them to enforce consistency between the mapping of two views of the same image. And instead of directly comparing the features of two images, like how contrastive learning does, they are comparing cluster assignments.

2 Method

2.1 Definitions

- Denote B as the batch size.
- Denote k as the number of "clusters".

- $Z = \{z_1, \dots, z_B\}$: The encodings of our input images. Each one of them is of dimension $(m, 1)$.
- $Q = \{q_1, \dots, q_B\}$: The codes or cluster assignments. They are recalculated for each batch, and are treated as our signals for training. Each one of them is of dimension $(k, 1)$.
- $C = \{c_1, \dots, c_k\}$: They are called the *prototypes*, and each one of them is a learned embedding vector for a cluster. Each one of them is of dimension $(m, 1)$.

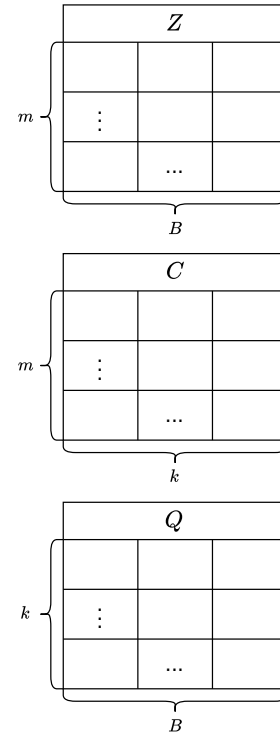


Figure 1: The dimensions of the encoder's outputs Z , the prototypes matrix C and the codes matrix Q .

2.2 Key Components

2.2.1 The codes Q . They represent the cluster assignment of each image in our batch: for each image x_i in our batch there is a corresponding q_i , which is a vector of size k , where each coordinate has a value between 0 and 1, that represents how much the image *belongs* to each cluster. The codes are treated as the signals for our pretext task. They are re-calculated for each batch with the following formula:

$$Q = \text{Diag}(u) \cdot \exp\left(\frac{C^T Z}{\epsilon}\right) \cdot \text{Diag}(v)$$

Where u and v are computed with the *Sinkhorn-Knopp* algorithm, to enforce an equal partition of the images to the clusters. They do this so the model does not learn the naive solution of putting all the images in one cluster.

2.2.2 The prototypes C . They are the vectors that we will use to map the images to clusters. So given a representation of an image z we can predict which cluster it belongs to by calculating the following:

$$p^{(k)} = \frac{\exp(1/\tau \cdot z_i^T \cdot c_k)}{\sum_{k'} \exp(1/\tau \cdot z_i^T \cdot c_{k'})}$$

Where p is a vector of size k such that each coordinate represents the probability that the image belongs to the corresponding cluster. This is basically taking softmax of the products of z and all the prototypes. Later this vector p is compared against the code q of another image, that comes from the same original image as z . We expect our model to maximize the similarity between p and q , because two views that come from the same image should belong to the same cluster.

2.2.3 The encodings Z . Produced by using different variants of ResNet50 as the encoders.

2.3 Swapping Assignments Between Views

The pretext task in contrastive learning is to give two different views of the same image similar representations, and in the clustering-based approaches it's to predict the cluster assignment of an image. But in SwAV we want the model to give two different views of the same image **similar cluster assignments**. And this is done by maximizing the similarity between the predicted cluster assignment of one view and the actual assignment of the other view (**the swap**).

So given two encodings z_t, z_s , and their codes q_t, q_s our loss function is:

$$L(z_t, z_s) = l(z_t, q_s) + l(z_s, q_t)$$

Where l is a function that measures the similarity between the actual assignment and a predicted assignment (Cross Entropy Loss):

$$l(z_t, q_s) = -q_s^T \cdot \log(p_t)$$

2.4 Multi-crop augmentation

In SimCLR random crop was one of multiple transformations that were used for their data augmentation, and in SwAV they mention that prior work notes on the role of comparing random crops in capturing information in images. But in this paper they take it up a notch. Rather than taking two random crops of an image and comparing them, they take $V + 2$ crops: 2 of them are standard resolution crops, and the rest are in low resolution, so the requirements in memory and compute do not increase by a lot. And therefore the actual loss that they computed is:

$$L(z_{t_1}, \dots, z_{t_{V+2}}) = \sum_{i \in \{1, 2\}} \sum_{v=1 \neq i}^{V+2} l(z_{t_v}, q_{t_i})$$

Note that they only use the full resolution crops to compute codes to save computations, and also they found out that doing this gives better results and explained it by the fact that these low resolution crops capture less information and therefore could degrade the quality of the assignments.

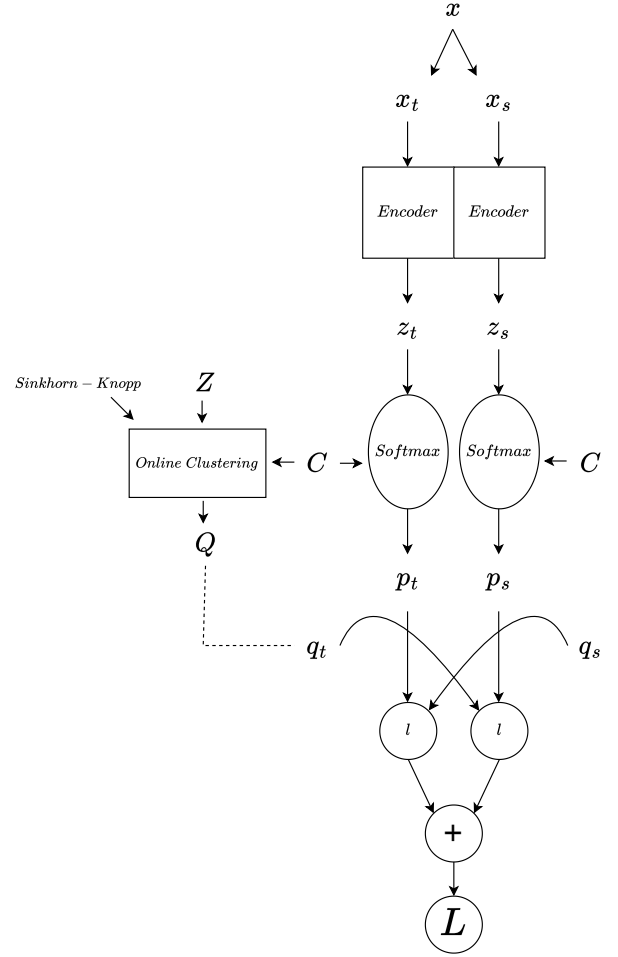


Figure 2: Overview of SwAV Self-Supervised Learning Architecture.

3 More Details

3.1 Soft Codes

The result of the online clustering method, done with Sinkhorn-Knopp, produces a continuous Q^* , they called them soft codes. They decided to keep the codes continuous as probabilities, instead of rounding them and turning them into hard codes, because it gives better performance. Their explanation to that is that the rounding is a more aggressive optimization step than gradient updates.

3.2 Working With Small Batches

A problem they encountered is how to deal with a situation where the batch size is smaller than the number of prototypes or clusters. It's a problem because in this situation it is not possible to equally partition the batch samples into the prototypes. They solve this by using samples from prior batches.

3.3 SimCLR's Influence

They note in this paper that they borrowed ideas from SimCLR such as using the MLP linear head.

4 A Look At the Results

We can see from **Table 1** that SwAV beats SimCLR easily with small batch sizes, and it also beats MoCo even with way less stored features (65, 536 vs. 3, 840). SwAV also outperforms supervised learning on multiple object detection sets, and with different architectures.

The most notable result they showed is that SwAV beat supervised learning on *Places205*, *VOC07* and *iNat18*, making it the first ever self-supervised learning method to ever do that! (**Table 2**)

An interesting experiment they did was trying out different clustering-based and contrastive learning methods with the multi-crop augmentation strategy, and they found that clustering methods benefited more from it than contrastive methods did. SimCLR, for instance, only saw a +2% improvement in accuracy with the linear evaluation protocol. A clustering-based method called *DeepCluster-v2* actually beats SwAV both with and without multi-crop, but they note that this method is not online.

5 Critique

I have the same criticism as in my previous analysis: it would have been nice to see them experiment with combinations of different transformations for the augmentation, especially since in the SimCLR paper they displayed the importance of combining augmentations from two different transformation types: spatial and appearance.

Method	epochs	batch	Top1 acc.
MoCo	200	256	60.6
SimCLR	200	256	61.9
SimCLR	200	8192	66.6
MoCo v2	200	256	67.5
MoCo v2	800	256	71.1
SwAV	200	256	72.7
SwAV	400	256	74.3

Table 1: Comparison of SwAV, SimCLR and MoCo on the linear evaluation protocol with ImageNet, using ResNet50 as the backbone.

6 Back To The False Negatives Problem

Although this problem is not addressed in the paper as an advantage of SwAV, I wanted to note that this is not a problem here. Because

Method	Places205(Top1)	VOC07(mAP)	iNat18(tTop1)
Supervised	53.2	87.6	46.7
SwAV	56.7	88.9	48.6

Table 2: Comparison of SwAV and supervised learning on transferring to linear classification datasets, using ResNet50 as the backbone.

SwAV is contrasting between the clustering assignment, and is not trying to attract positives and repel negatives, the false negative problem is not present in SwAV.

References

- [1] Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. Unsupervised learning of visual features by contrasting cluster assignments. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9912–9924, 2020.